

Guide de Maintenance — DataShare

Vue d'ensemble

Ce document décrit les procédures de maintenance de la plateforme DataShare, incluant les mises à jour des dépendances, les sauvegardes, le rollback, la surveillance et la réponse aux incidents.

1. Gestion des dépendances

Backend (NestJS)

```
cd backend

# Vérifier les paquets obsolètes
npm outdated

# Audit de sécurité
npm audit

# Corriger les vulnérabilités non-cassantes
npm audit fix

# Mettre à jour toutes les dépendances (mineures/correctifs)
npm update

# Mettre à jour un paquet spécifique
npm install @nestjs/core@latest
```

Planification : Exécuter `npm audit` chaque semaine. Appliquer les correctifs de sécurité dans les 48h pour les sévérités critiques/hautes.

Fréquence de mise à jour des dépendances et risques

Dépendance	Version actuelle	Fréquence de mise à jour	Niveau de risque	Notes
NestJS (@nestjs/*)	10.x	Mineure : mensuelle, Majeure : ~annuelle	 Moyen	Les versions majeures peuvent nécessiter un guide de migration. Tester toutes les routes après mise à jour.
Prisma	5.x	Mineure : bimensuelle, Majeure : ~annuelle	 Élevé	Les changements de schéma peuvent nécessiter prisma generate + migration. Toujours sauvegarder la BDD avant.
React	18.x	Mineure : mensuelle, Majeure : ~2 ans	 Moyen	Les mises à jour majeures (ex. 18→19) peuvent casser les

				hooks/cycle de vie. Tester toutes les pages.
Vite	5.x	Mineure : mensuelle, Majeure : ~annuelle	 Faible	Généralement non-cassant. Changements de config possibles en majeure.
@aws-sdk/client-s3	3.x	Correctif : hebdomadaire, Mineure : mensuelle	 Faible	API stable. Surveiller les avis de dépréciation.
bcrypt	5.x	Rare	 Faible	Module natif — peut nécessiter une recompilation lors d'une mise à jour majeure de Node.js.
class-validator	0.14.x	Irrégulière	 Moyen	Pré-1.0 — les décorateurs peuvent changer. Figurer la version.
Playwright	1.x	Mineure : bimensuelle	 Faible	Dev uniquement. Binaires navigateur téléchargés automatiquement.
Jest	29.x	Mineure : mensuelle	 Faible	Dev uniquement. Rarement cassant.

Politique de mise à jour

Type	Fréquence	Procédure	Approbation
Correctifs de sécurité (critique/élevé)	Sous 48h	npm audit fix → exécuter les tests → déployer	Responsable technique
Mises à jour correctives (x.y.Z)	Hebdomadaire	npm update → exécuter les tests	Développeur
Mises à jour mineures (x.Y.0)	Mensuelle	Mettre à jour une par une → suite de tests complète	Développeur
Mises à jour majeures (X.0.0)	Revue trimestrielle	Branche dédiée → guide de migration → E2E complet	Responsable technique + revue

Risques à surveiller

Risque	Impact	Atténuation
Incompatibilité du schéma Prisma	Corruption de la BDD, perte de données	Toujours sauvegarder avant la migration, tester en staging
Mise à jour majeure de Node.js	Les modules natifs (bcrypt) peuvent casser	Tester d'abord dans Docker, recompiler les dépendances natives
Changements cassants de React	Régression de l'interface	Exécuter la suite E2E complète (21 tests Playwright)

Attaque de la chaîne d'approvisionnement npm	Dépendance compromise	Figurer les versions dans package-lock.json, exécuter npm audit
Expiration du certificat TLS	HTTPS cassé	Mettre un rappel dans le calendrier, automatiser avec certbot (prod)

Frontend (React/Vite)

```
cd frontend

# Générer le lockfile s'il est manquant
npm i --package-lock-only

# Vérifier les vulnérabilités
npm audit

# Mettre à jour les dépendances
npm update
```

Prisma (ORM)

```
cd backend

# Mettre à jour le CLI Prisma + le client
npm install prisma@latest @prisma/client@latest

# Régénérer le client après des changements de schéma
npx prisma generate

# Appliquer les changements de schéma à la base de données
npx prisma db push      # Développement (pas de fichiers de migration)
npx prisma migrate dev  # Production (crée des fichiers de migration)
```

⚠ **Important** : Toujours sauvegarder la base de données avant d'exécuter des migrations en production.

2. Sauvegarde et restauration de la base de données

Sauvegarde PostgreSQL

```
# Dump complet de la base de données (depuis Docker)
docker exec datashare-postgres pg_dump -U datashare datashare > backup_$(date +%Y%m%d_%H%M%S).sql

# Sauvegarde compressée
docker exec datashare-postgres pg_dump -U datashare datashare | gzip > backup_$(date +%Y%m%d).sql.gz
```

Restauration PostgreSQL

```
# Restaurer depuis un dump
cat backup_20260618.sql | docker exec -i datashare-postgres psql -U datashare datashare

# Restaurer depuis une sauvegarde compressée
gunzip -c backup_20260618.sql.gz | docker exec -i datashare-postgres psql -U datashare datashare
```

Sauvegarde MinIO

```
# En utilisant mc (Client MinIO)
docker run --rm -v $(pwd)/minio-backup:/backup \
  --network infra_datashare-net \
  minio/mc mirror minio/datashare /backup/

# Ou copier directement le volume Docker
docker cp datashare-minio:/data ./minio-backup
```

Planification des sauvegardes (Recommandée)

Données	Fréquence	Rétention	Méthode
PostgreSQL	Quotidienne	30 jours	pg_dump + cron
Fichiers MinIO	Quotidienne	30 jours	mc mirror + cron
Configuration .env	À chaque changement	Historique Git	Sauvegarde chiffrée

3. Déploiement et rollback

Déploiement standard

```
cd infra

# Récupérer les dernières images / reconstruire
docker compose build --no-cache

# Déployer sans interruption (recréer un par un)
docker compose up -d --force-recreate

# Vérifier la santé
curl -k https://localhost/api/health
```

Procédure de rollback

```
# 1. Identifier la dernière version fonctionnelle
git log --oneline -5

# 2. Basculer sur la version fonctionnelle
git checkout v0.5.4 # ou un hash de commit spécifique

# 3. Reconstruire et redéployer
cd infra && docker compose build && docker compose up -d

# 4. Vérifier
curl -k https://localhost/api/health

# 5. Si le rollback est stable, créer une branche de correctif
git checkout -b hotfix/rollback-from-v0.6.0
```

Rollback de la base de données

```
# Si une migration Prisma a causé des problèmes
cd backend && npx prisma migrate reset # ⚠ DÉTRUIT LES DONNÉES

# Plus sûr : restaurer depuis une sauvegarde
cat backup_before_migration.sql | docker exec -i datashare-postgres psql -U
datashare datashare
```

4. Surveillance et vérifications de santé

Endpoint de santé

```
# Vérification de santé du backend
curl -k https://localhost/api/health
# Attendu : {"status": "ok"}
```

Santé des conteneurs Docker

```
# Vérifier le statut de tous les conteneurs
docker compose -f infra/docker-compose.yml ps

# Vérifier les journaux d'un service spécifique
docker compose -f infra/docker-compose.yml logs --tail=50 backend
docker compose -f infra/docker-compose.yml logs --tail=50 postgres
docker compose -f infra/docker-compose.yml logs --tail=50 minio

# Suivre les journaux en temps réel
docker compose -f infra/docker-compose.yml logs -f backend
```

Métriques clés à surveiller

Métrique	Source	Seuil d'alerte
Redémarrages de conteneurs	<code>docker ps</code>	> 3 en 5 minutes
Temps de réponse de l'API	Journaux d'accès / k6	p95 > 2000ms
Utilisation disque (MinIO)	<code>docker system df</code>	> 80%
Connexions à la base de données	Journaux PostgreSQL	> 80% de <code>max_connections</code>
Taux d'erreur	Journaux backend	> 5% des requêtes
Utilisation mémoire	<code>docker stats</code>	> 90% de la limite

Commandes de diagnostic rapide

```
# Utilisation des ressources par conteneur
docker stats --no-stream

# Utilisation disque par volume
docker system df -v

# Connexions actives PostgreSQL
docker exec datashare-postgres psql -U datashare -c "SELECT count(*) FROM pg_stat_activity;"

# Taille du bucket MinIO
docker exec datashare-minio mc du local/datashare
```

5. Problèmes courants et corrections

Backend en boucle de redémarrage

Symptôme : `docker ps` affiche le backend avec le statut "Restarting"

Diagnostic :

```
docker compose -f infra/docker-compose.yml logs --tail=30 backend
```

Causes courantes :

Erreur	Cause	Correction
EPROTO ssl3_get_record	MINIO_USE_SSL=false interprété comme chaîne vraie	Définir comme chaîne 'false', corriger dans le code : comparaison == 'true'
ECONNREFUSED postgres:5432	PostgreSQL pas encore prêt	Ajouter <code>depends_on</code> + <code>healthcheck</code> dans <code>docker-compose</code>
JWT_SECRET must be configured	Fichier <code>.env</code> manquant	Copier <code>.env.example</code> vers <code>.env</code> et remplir les valeurs

Prisma: table not found	Base de données non initialisée	Exécuter <code>npx prisma db push</code> dans le conteneur backend
-------------------------	---------------------------------	--

Échec du téléversement de fichier

Erreur	Cause	Correction
413 Payload Too Large	Limite de taille du corps Nginx	Définir <code>client_max_body_size 1g;</code> dans <code>nginx.conf</code>
MinIO connection refused	Conteneur MinIO non démarré	<code>docker compose up -d minio</code>
SignatureDoesNotMatch	Incompatibilité URL interne vs publique	Définir <code>MINIO_PUBLIC_URL</code> correctement

Problèmes d'authentification

Erreur	Cause	Correction
401 sur toutes les routes	JWT expiré	Se reconnecter ou vérifier <code>JWT_EXPIRES_IN</code>
Échec du rafraîchissement du jeton	Jeton révoqué ou expiré	Se reconnecter
Cookie non envoyé	Incompatibilité SameSite/Secure	S'assurer que HTTPS + configuration correcte des cookies

6. Liste de vérification du déploiement

Avant chaque déploiement, vérifier :

- ☐ Tous les tests unitaires passent : `cd backend && npm test`
- ☐ Les tests E2E passent (si la pile est en cours d'exécution) : `cd e2e && npx playwright test`
- ☐ Le fichier `.env` contient toutes les variables requises (comparer avec `.env.example`)
- ☐ Sauvegarde de la base de données effectuée
- ☐ `CHANGELOG.md` mis à jour
- ☐ Images Docker reconstruites : `docker compose build`
- ☐ L'endpoint de santé répond : `curl -k https://localhost/api/health`
- ☐ Le flux de téléversement/téléchargement de fichier fonctionne manuellement
- ☐ Aucun secret commité (vérification `git diff --cached`)

7. Contact et escalade

Niveau	Action	Qui
N1	Vérifier les journaux, redémarrer les conteneurs	Développeur d'astreinte

N2	Restauration de la base de données, rollback du déploiement	Responsable technique
N3	Changements d'infrastructure, incident de sécurité	CTO / Équipe infrastructure